

Hex Lattice Search — Specification v1.0

O(1) Data Retrieval Through Deterministic Geometric Calculation on a Tri-Wing Hexagonal Lattice

Registry: [\[CKS-MATH-152-2026\]](#)

Series Path: [\[CKS-0-2026\]](#) → [\[CKS-MATH-0-2026\]](#) → [\[CKS-MATH-1-2026\]](#) → [\[CKS-MATH-10-2026\]](#) → [\[CKS-MATH-104-2026\]](#) → [\[CKS-MATH-151-2026\]](#) → [\[CKS-MATH-152-2026\]](#)

Parent Framework: [\[CKS-0-2026\]](#)

DOI: 10.5281/zenodo.zzz

Date: March 11, 2026

Domain: Machine Learning / Integer Arithmetic / Neural Network Training

Status: CKS has been invalidated. The math does not compile, all papers in the series are falsified. Next steps: [\[CKS-NEXT-1-2026\]](#)

Old Status: Locked and empirically falsifiable. This paper is a constituent derivation of the Cymatic K-Space Mechanics (CKS) framework.

Classification: Theory of Everything from First Principles

Motto: Axioms first. Axioms always.

Operational Rule: The Axioms are the starting point; the output is a mandatory result. Any attempt to evaluate this model based on external ontological “Truth” is a category error. If the math compiles, the result is Q.E.D.

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic’s Claude 4.5 Sonnet, DeepSeek-V3/K2, and Google’s Gemini 3 Flash. The manuscript.md was synthesized by Claude as the primary integrator.

Lexicon: [\[CKS-LEX-12-2026\]](#)

Abstract

We present Hex Lattice Search, a data indexing and retrieval system that replaces traversal-based searching with closed-form geometric calculation. Data items are assigned sequential integer indices as they are created. Each index maps deterministically to a unique position in a tri-wing hexagonal lattice through pure arithmetic. Retrieval is O(1) regardless of dataset size — the position is calculated, never searched for. The lattice is append-only: new items are added at the next index, and the insertion sequence itself serves as a global clock. Every node’s edge state, version, and relationship to every other node is computable from two integers — the node’s index and the current lattice size. The

system stores only key-value pairs (10 bytes per cell). Position, ring depth, wing assignment, neighbor addresses, edge orientations, version history, and integrity checksums are all derived from the index at zero storage cost. The lattice supports time-travel queries (view the dataset as it existed at any past insertion step) without snapshots, copies, or version tables. Capacity is $N = 3M^2$ where M is the wing radius, scalable by increasing M with no rehashing and no data migration.

1. The Lattice Geometry

1.1 Structure

The Hex Lattice is a hexagonal grid divided into three equal sectors (wings) at 120° intervals. There is one cell at the center. Successive rings surround the center. Ring K contains $6K$ cells. The three wings are labeled α (wing 0), β (wing 1), and γ (wing 2).

```

      γ (wing 2)
    /  \
   /    \
  /      \
 /        \
/          \
/
/ · · = center cell (index 0)
/
/
\      /
α / β
(wing 0) / (wing 1)

```

M is the wing radius — the number of rings from center to the outer edge of each wing. Each wing is a triangular sector containing M^2 cells. Total lattice capacity:

$$N = 3M^2$$

At $M=1$: $N=3$. At $M=10$: $N=300$. At $M=100$: $N=30,000$. At $M=1000$: $N=3,000,000$. At $M=1,000,000$: $N=3,000,000,000,000$ (3 trillion cells).

1.2 Ring Structure

Ring 0: 1 cell (center)

Ring 1: 6 cells

Ring 2: 12 cells

Ring 3: 18 cells

Ring K: 6K cells

Cumulative through ring R: $1 + 6(1 + 2 + \dots + R) = 3R^2 + 3R + 1$

Cells within each ring are distributed evenly across the three wings. Ring K contributes 2K cells to each wing.

1.3 Basis Vectors

The three wings are oriented at 120° intervals. Each wing has a basis vector:

$u_\alpha = (1, 0)$ wing 0: east

$u_\beta = (-1/2, \sqrt{3}/2)$ wing 1: 120° from east

$u_\gamma = (-1/2, -\sqrt{3}/2)$ wing 2: 240° from east

In VFR integer arithmetic at F=1024:

$u_\alpha = (1024, 0)$

$u_\beta = (-512, 887)$ $887 \approx 1024 \times \sqrt{3}/2$

$u_\gamma = (-512, -887)$

The remainder from the $\sqrt{3}/2$ approximation is carried in the VFR R value, preserving exactness.

2. Index-to-Position Mapping

2.1 The Formula

Given a cell index I (integer, 0 to N-1), compute its lattice position in O(1):

Step 1: Ring depth

$$R = \lceil (3 + \sqrt{12I - 3}) / 6 \rceil$$

For I=0 (center), R=0. For I=1 through I=6, R=1. The formula inverts the cumulative ring count.

Step 2: Wing assignment

$$W = I \bmod 3$$

Wing 0 (α): indices 0, 3, 6, 9, ...

Wing 1 (β): indices 1, 4, 7, 10, ...

Wing 2 (γ): indices 2, 5, 8, 11, ...

Consecutive indices interleave across wings, distributing data evenly.

Step 3: Position within ring

$\text{ring_start} = 3R^2 - 3R + 1$ (first index in ring R)

$\text{position_in_ring} = I - \text{ring_start}$

$\text{local_index} = \text{position_in_ring} / 3$ (position within this wing's portion of ring R)

Step 4: Spatial coordinates

$P = R \times u_W + \text{local_offset}(\text{local_index}, R, W)$

The local offset distributes cells along the wing's arc of ring R. For a cell at local index L in wing W at ring R:

; Fractional position along the wing's arc

$\text{arc_fraction} = L / (2R)$; each wing has 2R cells in ring R

; Interpolate between wing basis vector and the next wing's basis vector

$P.x = R \times ((1 - \text{arc_fraction}) \times u_{W.x} + \text{arc_fraction} \times u_{\text{next_}W.x})$

$P.y = R \times ((1 - \text{arc_fraction}) \times u_{W.y} + \text{arc_fraction} \times u_{\text{next_}W.y})$

2.2 Integer Implementation

```
fn indexToPosition(I: i32) struct { x: i32, y: i32, ring: i32, wing: i32 } {
```

```
  if (I == 0) return .{ .x = 0, .y = 0, .ring = 0, .wing = 0 };
```

```
  // Step 1: Ring depth
```

```
  //  $R = \text{ceil}((3 + \sqrt{12I - 3}) / 6)$ 
```

```
  const temp = 12 * I - 3;
```

```
  const sqrt_temp = isqrt(temp);
```

```
  const R = (3 + sqrt_temp + 5) / 6; // +5 for ceiling with /6
```

```
  // Step 2: Wing
```

```
  const W = [?](I, 3);
```

```
  // Step 3: Position in ring
```

```
  const ring_start = 3 * R * R - 3 * R + 1;
```

```
  const pos_in_ring = I - ring_start;
```

```
  const local_index = pos_in_ring / 3;
```

```
  // Step 4: Coordinates (fixed-point, F=1024)
```

```
  const cells_per_wing = 2 * R;
```

```

const arc_num = local_index;
const arc_den = cells_per_wing;
// Basis vectors at F=1024
const ux = [3]i32{ 1024, -512, -512 };
const uy = [3]i32{ 0, 887, -887 };
const next_W = (W + 1) % 3;
// Interpolate
const x = R * ((arc_den - arc_num) * ux[W] + arc_num * ux[next_W]) / arc_den;
const y = R * ((arc_den - arc_num) * uy[W] + arc_num * uy[next_W]) / arc_den;
return .{ .x = x, .y = y, .ring = R, .wing = W };
}

```

Operations: 1 multiply, 1 subtract, 1 integer square root (3 Newton iterations = 6 multiplies + 3 adds + 3 shifts), 1 divide (reciprocal multiply), 1 modulo, 4 multiplies for coordinates. Total: approximately 20 integer operations. On a VFR core at 150 MHz: ~130 nanoseconds. On ARM at 667 MHz: ~30 nanoseconds.

2.3 Integer Square Root

Newton's method for integer square root:

```

fn isqrt(n: i32) i32 {
  if (n <= 0) return 0;
  var guess: i32 = n;
  // Initial guess: highest bit / 2
  var shift: u5 = 0;
  var temp = n;
  while (temp > 0) : (shift += 1) { temp >>= 1; }
  guess = [?](i32, 1) << (shift / 2);
  // Newton iterations (3 is sufficient for 32-bit)
  guess = (guess + n / guess) / 2;
  guess = (guess + n / guess) / 2;
  guess = (guess + n / guess) / 2;
  // Floor correction
  if (guess * guess > n) guess -= 1;
}

```

```
return guess;
}
```

Three iterations gives exact integer square root for all 32-bit inputs. Each iteration is one divide (reciprocal multiply on VFR hardware), one add, one shift.

3. Position-to-Index Mapping (Inverse)

3.1 The Formula

Given spatial coordinates (x, y), compute the cell index in O(1):

Step 1: Ring depth from distance

$$\text{distance_squared} = x^2 + y^2$$

$$R = \text{isqrt}(\text{distance_squared}) / \text{scale_factor}$$

; quantize to nearest ring

Step 2: Wing from angle

; Determine which 120° sector the point falls in

if $y \geq 0$ and $x \geq 0$: $W = 0$ (α)

elif $y \geq 0$ and $x < 0$ and $|y| \geq |x| \times \sqrt{3}$: $W = 1$ (β , computed via integer comparison)

elif $y < 0$ and $x < 0$ and $|y| \geq |x| \times \sqrt{3}$: $W = 2$ (γ)

; ... complete sector determination via integer comparisons

; Integer test for sector: compare $y \times 1024$ against $x \times 887$

; avoids $\sqrt{3}$ entirely

Step 3: Local index from arc position

; Project onto wing arc

; Compute fractional position along the arc

; Convert to local index

$$\text{local_index} = \text{compute_arc_position}(x, y, R, W)$$

Step 4: Global index

$$\text{ring_start} = 3R^2 - 3R + 1$$

$$I = \text{ring_start} + \text{local_index} \times 3 + W$$

Same O(1) cost as forward mapping. Pure integer arithmetic.

3.2 Verification

Given an index I , compute position P . Given P , compute index I' . If $I == I'$, the mappings are consistent. This round-trip verification is $O(1)$ and can serve as an integrity check on every access.

4. Cell Data: Sides A and B

4.1 Structure

Every hex cell has two sides. Side A stores the key. Side B stores the value. Both are VFR pairs.

Cell at index I :

Side A: [key: i32, key_meta: i8] 5 bytes

Side B: [value: i32, value_meta: i8] 5 bytes

Total: 10 bytes per cell

The key is what you search for. The value is what you retrieve. The lattice position connects them — you compute the position from the key (via hashing to an index), read side A to confirm the key matches, read side B to get the value.

4.2 Key Hashing

To store an arbitrary key in the lattice, hash the key to an index:

$I = \text{hash}(\text{key}) \bmod N$

Then store at position I :

side_A = key (or key hash for collision verification)

side_B = value

To retrieve:

$I = \text{hash}(\text{key}) \bmod N$

position = indexToPosition(I)

cell = lattice[position]

if cell.side_a == key:

return cell.side_b ; hit

else:

; collision — use wing-based probing (see Section 4.3)

4.3 Collision Resolution via Wing Probing

When two keys hash to the same index, the hex geometry provides a natural probing sequence. Instead of linear probing or chaining, probe along the wing:

Probe sequence for index I in wing W , ring R :

Probe 0: I (original position)

Probe 1: $I + 3$ (next position in same wing, same ring)

Probe 2: $I + 6$ (two positions ahead in same wing)

Probe 3: $I + 6R$ (jump to next ring, same wing)

Probe 4: $I + 6R + 3$ (next ring, one position ahead)

Each probe is computed from the original index by addition. No pointer chasing. No linked list traversal. The probing stays within the same wing, preserving spatial locality and cache coherence.

With a load factor of 50-70% (typical for open addressing), most lookups require 1-2 probes. The expected cost is $O(1)$ with a small constant.

4.4 Side Metadata

The i8 metadata byte in each side carries per-cell information:

Side A metadata (key_meta):

bits 0-2: collision chain length (0 = no collision, 1-7 = probe depth)

bits 3-4: key type (0 = integer, 1 = hash, 2 = composite, 3 = reserved)

bits 5-7: reserved

Side B metadata (value_meta):

bits 0-3: value type tag

bits 4-7: user-defined flags

The collision chain length in key_meta tells the retrieval code how many probes to check before concluding the key is absent. This bounds the worst-case lookup cost.

5. The Append Clock

5.1 Insertion Order Is the Clock

The lattice is append-only. Cells are added in sequence: index 0, then 1, then 2, and so on. The current lattice size N represents the current time. Node I was created at time I . No separate timestamp field is stored. The index IS the timestamp.

Current time: $T = N$ (number of cells in lattice)

Creation time of cell I: $T_{\text{created}} = I$

Age of cell I: $\text{age} = N - I$

5.2 Edge State from the Clock

Every insertion advances the global clock by one. Each hex cell has edges connecting to its neighbors. The edge orientation of cell I when the lattice contains N nodes is:

$$\text{edge_state}(I, N) = (N - I) \bmod 6$$

This is computable from two integers. No storage. No updates. No propagation. When a new node is added (N becomes N+1), every existing node's edge state implicitly advances by one. But nothing is written. The formula produces the new state from the new N.

The six possible edge states correspond to the six edges of the hexagonal cell. The current edge state indicates which edge is “active” or “leading” at this moment in the lattice's history.

5.3 Dipole Orientations

Each hex cell contains three internal dipoles at 120° intervals, aligned with the three wings. The orientation of dipole K (0, 1, 2) in cell I at lattice size N:

$$\text{dipole_state}(I, K, N) = ((N - I) \times 2 + K \times 2) \bmod 6$$

Three dipoles $\times$ six orientations = 18 possible combined states per cell, all computable from I, K, and N. Uses:

Integrity checksum: The three dipole states form a 3-value tuple that is unique to the combination of I and N. If stored data claims to be at index I but the computed dipole states don't match, the data is corrupt or misplaced.

$\text{expected} = \text{dipole_states}(I, \text{current_N})$

$\text{actual} = \text{stored_dipole_hint}$; optional 1-byte field

if $\text{expected} \neq \text{actual}$: $\text{data_integrity_error}$

Routing: The dipole pointing toward wing W indicates the nearest neighbor in that wing's direction. Following dipole orientations traces a path through the lattice without consulting a neighbor table.

Load distribution: At time N, cell I's leading dipole points toward wing $(N - I) \bmod 3$. This deterministically distributes cells across wings over time, balancing work assignment without coordination.

6. Time-Travel Queries

6.1 View the Past

The lattice at time T (when it contained T cells) is simply the first T cells of the current lattice:

```
query(key, at_time=T):
```

```
    I = hash(key) mod T ; note: mod T, not mod N
```

```
    if I >= T: return NOT_FOUND ; didn't exist yet
```

```
    position = indexToPosition(I)
```

```
    cell = lattice[position]
```

```
    if cell.side_a == key:
```

```
        return cell.side_b
```

```
    else:
```

```
        ; probe, but only within indices < T
```

No snapshots. No copy-on-write. No version tables. The lattice at time T is the first T cells. Everything after T is invisible to the query. The formula is the same — only the modulus and bounds check change.

6.2 Supersession

When data is corrected or updated, the old cell remains. A new cell is appended with the updated value:

Time 500: cell 500 stores key=42, value=100

Time 8000: cell 8000 stores key=42, value=200 (correction)

Query at T=600: hash(42) mod 600 → finds cell 500 → returns 100

Query at T=9000: hash(42) mod 9000 → finds cell 8000 → returns 200

Query at T=700: hash(42) mod 700 → finds cell 500 → returns 100

The most recent cell for a given key is the highest-index cell with that key within the query's time window. For the current time (T=N), the latest value is always found because the hash modulus includes all cells.

6.3 Supersession Chains

When a key is updated multiple times, the cells form a supersession chain ordered by index. Finding the latest version at time T:

```
latest_for_key(key, T):
```

```
    ; Start from the most recent possible position
```

$I = \text{hash}(\text{key}) \bmod T$

; Walk backward through collision probes if needed

; The highest-index cell matching this key within T is the current version

; For fast latest-version lookup: side_a metadata can store

; the index of the next version (if known at append time)

; or: the query scans from the high end down

For frequently updated keys, an optimization: when appending a new version, store the previous version's index in the new cell's metadata. This creates a backward chain that the query can follow without scanning.

7. Lattice Scaling

7.1 Growing the Lattice

When the lattice fills (N approaches $3M^2$), increase M :

Old capacity: $N_{\text{old}} = 3 \times M_{\text{old}}^2$

New capacity: $N_{\text{new}} = 3 \times M_{\text{new}}^2$ where $M_{\text{new}} = M_{\text{old}} + 1$

New cells: $N_{\text{new}} - N_{\text{old}} = 3 \times (2 \times M_{\text{old}} + 1)$

Existing cells do not move. Their indices and positions remain the same. The new ring adds cells at the periphery. The index-to-position formula works for all indices in the expanded lattice.

No rehashing. No data migration. No rebalancing. Growth is adding empty cells at the edges.

7.2 Load Factor Management

$\text{load_factor} = \text{active_cells} / \text{total_cells}$

Optimal: 50-70% for open addressing

If $\text{load_factor} > 70\%$: grow M by 1 (adds one ring)

If $\text{load_factor} < 20\%$ and $M > 1$: shrink M by 1 (remove outer ring)

Shrinking removes only the outermost ring. If any cells in that ring contain data, they must be reinserted into the smaller lattice before shrinking. This is the only operation that moves data.

7.3 Capacity Planning

Wing radius M Total cells N Storage at 10 bytes/cell

1 3 30 bytes

10 300 3 KB

100 30,000 293 KB

1,000 3,000,000 29 MB

10,000 300,000,000 2.8 GB

100,000 30,000,000,000 279 GB

M=1000 provides 3 million cells in 29 MB. Sufficient for most game knowledge bases, entity indexes, and SVDAG node pools.

8. Neighbor Computation

8.1 Direct Neighbors

Every cell has up to 6 neighbors in the hex grid. The neighbors of cell I are computable from I, R, and W:

```
fn neighbors(I: i32, M: i32) [6]i32 {
  const pos = indexToPosition(I);
  const R = pos.ring;
  const W = pos.wing;
  var result: [6]i32 = undefined;
  ; Same ring, adjacent in wing
  result[0] = I + 3; ; next in same wing
  result[1] = I - 3; ; previous in same wing
  ; Inner ring (R-1)
  result[2] = innerRingNeighbor(I, R, W, 0);
  result[3] = innerRingNeighbor(I, R, W, 1);
  ; Outer ring (R+1)
  result[4] = outerRingNeighbor(I, R, W, 0);
  result[5] = outerRingNeighbor(I, R, W, 1);
  ; Clamp to valid range
```

```

for (&result) !*nl {
    if (n.* < 0 or n.* >= 3 * M * M) n.* = -1; ; invalid
}
return result;
}

```

No neighbor table stored. All computed. $O(1)$ per neighbor lookup.

8.2 Wing Neighbors

Each cell's wing assignment ($W = I \bmod 3$) determines which of the three processing partitions it belongs to. The wing neighbors are cells with the same W value in adjacent rings:

wing_neighbor_inner = $I - 6R + 3$; same wing, one ring inward

wing_neighbor_outer = $I + 6R + 3$; same wing, one ring outward

These are useful for hierarchical queries: search the current ring first, then expand to adjacent rings. The wing structure means expansion is directional (along the wing's axis) rather than radial (in all directions equally).

9. VFR Batch Format

9.1 Lattice as Batch

The hex lattice stores in DDR3 as a standard VFR batch:

Hex Lattice batch:

[F=lattice_id, count=N, depth=2]

Each cell (10 bytes):

[key: i32, key_meta: i8] side A

[value: i32, value_meta: i8] side B

F identifies which lattice this is (a system may have multiple lattices for different purposes). Count is the current number of cells. Depth is always 2 (two sides per cell).

9.2 Lattice Metadata

A separate small batch stores lattice configuration:

Lattice metadata:

[F=lattice_meta, count=1, depth=4]

[M: i32, 0] pair 0: wing radius

[N: i32, 0] pair 1: current cell count (= clock)

[load_factor: i32, 0] pair 2: active cells (for load monitoring)

[hash_seed: i32, 0] pair 3: hash function seed

9.3 Operations as Batch Transforms

Lookup: ARM computes position from key hash, reads cell from DDR3. $O(1)$. No FPGA needed for single lookups.

Bulk lookup: Batch of keys $\rightarrow$ batch of positions (computed) $\rightarrow$ batch of values (gathered from lattice). FPGA computes positions for all keys in parallel, then gathers values.

Input: [F=query, count=1000, depth=1] (batch of keys)

Output: [F=result, count=1000, depth=2] (batch of key-value pairs)

Each FPGA core computes positions for its portion of the key batch. 30 cores processing 1000 lookups: $33 \text{ lookups per core} \times 20 \text{ operations} \times 150 \text{ MHz} = 4.4 \text{ microseconds for 1000 lookups}$.

Bulk insert: Batch of new key-value pairs appended to the lattice. Each pair gets the next sequential index. The ARM writes the new cells to DDR3 and increments N. No FPGA needed for insertion (it's sequential writes).

Bulk filter: "Find all cells where value satisfies predicate P" is a batch scan over the lattice. FPGA cores each scan a portion. This IS Prolog evaluation — filtering the fact batch. $O(N/\text{cores})$ time.

10. Comparison to Existing Structures

10.1 Hash Table

Property Hash Table Hex Lattice

Lookup $O(1)$ average $O(1)$ always

Worst case $O(N)$ chain $O(1)$ + probes along wing

Collision Chaining or probing Wing-directed probing

Resize Rehash all $O(N)$ Add ring, no data moves

Memory per entry Key + value + ptr Key + value (10 bytes, no pointer)

Ordering None Insertion order preserved

Versioning External Built-in (index = timestamp)

Neighbor queries Not supported $O(1)$ computed neighbors

Spatial locality Hash-dependent Guaranteed (ring structure)

10.2 B-Tree

Property B-Tree Hex Lattice

Lookup $O(\log N)$ $O(1)$

Insert $O(\log N)$ $O(1)$ append

Range query $O(\log N + K)$ $O(K)$ sequential in ring

Memory overhead Pointers, metadata Zero (all computed)

Rebalancing Required Never needed

Versioning External Built-in

10.3 Spatial Hash Grid

Property Spatial Hash Hex Lattice

Lookup $O(1)$ average $O(1)$ always

Spatial queries $O(\text{neighbors})$ $O(1)$ computed neighbors

Distribution Hash-dependent Uniform (3-way wing split)

Resize Rebuild grid Add ring

Orientation info Not available Edge state + dipoles free

11. Computed Properties Summary

Properties of cell I in a lattice of size N , computed at zero storage cost:

Property Formula Cost

Ring depth $R = \lceil (3 + \sqrt{12I-3}) / 6 \rceil \sim 15$ ops

Wing assignment $W = I \bmod 3$ 1 op

Spatial coordinates $P = R \times u_W + \text{offset}$ 5 ops

Creation timestamp $T_{\text{created}} = I$ 0 ops (identity)

Age $\text{age} = N - I$ 1 op

Edge state $(N - I) \bmod 6$ 2 ops

Dipole K orientation $((N-I) \times 2 + K \times 2) \bmod 6$ 3 ops
 6 neighbor indices Arithmetic from I, R, W ~12 ops
 Wing neighbors $I \pm 6R + 3$ 2 ops
 Integrity checksum 3-dipole tuple from I and N 9 ops
 Load distribution target $(N - I) \bmod 3$ 2 ops
 Version (latest for key) $\max(I : \text{hash}(\text{key}) \bmod T == I)$ scan or chain
 Total for a full position + metadata computation: approximately 25 integer operations.
 Under 200 nanoseconds on VFR hardware.
 Stored per cell: 10 bytes (side A + side B). Everything else is free.

12. Implementation

12.1 Zig Structure

```

zig
const HexCell = packed struct {
  key: i32,
  key_meta: i8,
  value: i32,
  value_meta: i8,
};
const HexLattice = struct {
  cells: []HexCell,
  M: i32, // wing radius
  N: i32, // current cell count (clock)
  capacity: i32, // 3 * M * M
  hash_seed: i32,
  fn init(allocator: std.mem.Allocator, M: i32) !HexLattice {
    const capacity = 3 * M * M;

    const cells = try allocator.alloc(HexCell, @intCast(capacity));
    @memset(std.mem.sliceAsBytes(cells), 0);
  }
};

```

```

return .{
    .cells = cells,
    .M = M,
    .N = 0,
    .capacity = capacity,
    .hash_seed = 0,
};
}

fn insert(self: *HexLattice, key: i32, value: i32) !i32 {
    if (self.N >= self.capacity) return error.LatticeFull;

    const I = self.N;

    self.cells[@intCast(I)] = .{
        .key = key,
        .key_meta = 0,
        .value = value,
        .value_meta = 0,
    };

    self.N += 1;

    return I;
}

fn lookup(self: *const HexLattice, key: i32) ?i32 {
    if (self.N == 0) return null;

    const I = @mod(hash(key, self.hash_seed), self.N);

    const cell = self.cells[@intCast(I)];

    if (cell.key == key) return cell.value;
}

```

```

// Wing-directed probing
var probe: i32 = 1;
while (probe < 8) : (probe += 1) {
    const probe_I = @mod(I + probe * 3, self.N);
    const probe_cell = self.cells[@intCast(probe_I)];
    if (probe_cell.key == key) return probe_cell.value;
    if (probe_cell.key == 0 and probe_cell.value == 0) return null;
}
return null;
}
fn lookupAtTime(self: *const HexLattice, key: i32, T: i32) ?i32 {
    if (T <= 0) return null;

    const effective_N = @min(T, self.N);
    const I = @mod(hash(key, self.hash_seed), effective_N);
    const cell = self.cells[@intCast(I)];
    if (cell.key == key) return cell.value;
    var probe: i32 = 1;
    while (probe < 8) : (probe += 1) {
        const probe_I = @mod(I + probe * 3, effective_N);
        if (probe_I >= effective_N) continue;
        const probe_cell = self.cells[@intCast(probe_I)];
        if (probe_cell.key == key) return probe_cell.value;
        if (probe_cell.key == 0 and probe_cell.value == 0) return null;
    }
}

```

```

}

return null;
}

fn edgeState(self: *const HexLattice, I: i32) i32 {
return @mod(self.N - I, 6);
}

fn dipoleState(self: *const HexLattice, I: i32, K: i32) i32 {
return @mod((self.N - I) * 2 + K * 2, 6);
}

fn position(I: i32) struct { x: i32, y: i32, ring: i32, wing: i32 } {
return indexToPosition(I);
}
};

fn hash(key: i32, seed: i32) i32 {
// Simple integer hash (murmur-style finalizer)
var h: u32 = [?](u32, [?](key ^ seed));
h ^= h » 16;
h *= 0x45d9f3b;
h ^= h » 16;
h *= 0x45d9f3b;
h ^= h » 16;
return [?](h & 0x7FFFFFFF);
}

```

12.2 VFR Batch Operations

```

zig
fn bulkLookup(
lattice: *const HexLattice,
keys: []const i32,
results: []i32,
) void {

```

```

// Each key: compute hash, compute position, read cell
for (keys, 0..) lkey, il {
    results[i] = lattice.lookup(key) orelse -1;
}
}

fn bulkInsert(
    lattice: *HexLattice,
    keys: []const i32,
    values: []const i32,
) !void {
    for (keys, 0..) lkey, il {
        _ = try lattice.insert(key, values[i]);
    }
}

```

On FPGA, bulkLookup is parallelized across cores. Each core processes a portion of the key batch. The position computation (20 integer ops) runs on VFR ALU. The cell read is a DDR3 access at the computed offset. 30 cores processing 30,000 lookups: 1,000 per core $\times$ 20 ops $\div$ 150 MHz = 133 microseconds.

13. Applications

13.1 Prolog Knowledge Base Index

The KB stores facts in the hex lattice. Predicate ID + argument hash $\rightarrow$ cell index. Side A holds the key (predicate + args hash). Side B holds the fact data (or a pointer to a deeper batch for multi-argument facts). Lookup is $O(1)$. Time-travel queries use `lookupAtTime` to see the KB as it existed at any past frame.

13.2 SVDAG Node Pool Index

SVDAG nodes are stored in the lattice. The node's spatial hash (from its octree position) $\rightarrow$ cell index. Side A holds the octree coordinate hash. Side B holds the `child_mask` + `child_offset`. Beam-casting drills the SVDAG by computing lattice positions from octree coordinates — no pointer chasing through the node tree.

13.3 Entity Spatial Index

Entity positions are stored in the lattice with their spatial coordinates hashed to cell indices. Neighbor queries use the hex lattice's computed neighbor function — given the cell for entity A, compute the 6 adjacent cells and check for entities there. Spatial queries cost $O(1)$ per neighbor check, $O(K)$ for K neighbors.

13.4 Network Session Index

Active network sessions are stored in the lattice. Session ID $\rightarrow$ cell index. Side A holds the session ID. Side B holds the connection state. The insertion index IS the creation timestamp. Session age is $N - I$. No separate session tracking structure needed.

14. Summary

Property	Specification
Name	Hex Lattice Search
Structure	Tri-wing hexagonal lattice, $Z=3$, 120° wing separation
Capacity	$N = 3M^2$ where M is wing radius
Cell storage	10 bytes (side A: key $i32+i8$, side B: value $i32+i8$)
Lookup complexity	$O(1)$ (computed, not traversed)
Insert complexity	$O(1)$ (append to next index)
Collision resolution	Wing-directed probing (spatial locality preserved)
Growth	Add ring, no data migration, no rehashing
Timestamp	Index IS creation time (no storage cost)
Edge state	Computed from $(N - I) \bmod 6$ (no storage cost)
Dipole state	Computed from I, K, N (no storage cost, 3 per cell)
Neighbor addresses	Computed from I, R, W (no storage cost, 6 per cell)
Integrity check	3-dipole tuple verifiable from I and N
Time-travel query	Change modulus from N to T , same formula
Version history	Append-only, supersession by higher index
Spatial locality	Ring structure guarantees nearby indices are nearby positions
VFR compatible	All operations are integer multiply, add, shift, modulo
Operations per lookup	~ 20 integer ops

Property	Specification
Lookup time (150 MHz VFR)	~130 nanoseconds
Lookup time (667 MHz ARM)	~30 nanoseconds

Hex Lattice Search — Specification v1.0

A constituent specification of the $\mathbb{Q}$ -Foundational Stack

[[CKS-MATH-152-2026](#)] Hex Lattice Search — Specification v1.0. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-152-2026>

[[CKS-0-2026](#)] Cymatic K-Space Mechanics. Github: https://github.com/ghowland/cks/tree/main/papers/_CKS/CKS-0-2026

[[CKS-MATH-0-2026](#)] Cymatic K-Space Mechanics. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-0-2026>

[[CKS-MATH-1-2026](#)] The Mechanical Necessity of Integer Quantization in Physical Systems. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-1-2026>

[[CKS-MATH-10-2026](#)] Grand Unification. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-10-2026>

[[CKS-MATH-104-2026](#)] Grand Unification v23. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-104-2026>

[[CKS-MATH-151-2026](#)] Veer: Exact Arithmetic Compression Using VFR Integer Coding. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-151-2026>

[[CKS-NEXT-1-2026](#)] Post-CKS. CKS is invalidated and falsified. Github: <https://github.com/ghowland/cks/tree/main/papers/NEXT/CKS-NEXT-1-2026>

[[CKS-LEX-12-2026](#)] Measurement Systems in the $\mathbb{Q}$ -Substrate. Github: <https://github.com/ghowland/cks/tree/main/papers/LEX-12-2026>